

Reproducing a CodonBERT Approach for Gene Expression Prediction

Seyede Bahar Marashi

Hasti Jahantigh

Final Project, Introduction to Bioinformatics

Fall 2025

Abstract

This project studies the idea presented in the paper *CodonBERT: Using BERT for Sentiment Analysis to Better Predict Genes with Low Expression*. The paper treats gene expression categories in a way similar to sentiment labels and then uses the learned model for expression prediction. Our goal was to reproduce the main workflow for one organism and compare our results with the values reported in the paper. We used coding sequences and expression counts for *Saccharomyces cerevisiae*. Because training a BERT model from scratch was not practical with our available Google Colab resources, we used an existing pre-trained CodonBERT checkpoint and fine-tuned it first as a three-class classifier and then as a regression model. The classifier reached a macro F1-score of 0.667 and an AUROC of 0.831 on the validation set. The regression model reached a Spearman correlation of 0.863 on the test split. These values show that the workflow learned useful information from codon sequences. However, our setup is an adapted reproduction rather than an exact reproduction because the pretrained checkpoint, data preparation, and evaluation split are not identical to the original paper.

Keywords: CodonBERT, BERT, codon sequence, gene expression, classification, regression

1 Introduction

The expression level of a gene is affected by information in its sequence. The reviewed paper uses a BERT-based model to learn from codon sequences and predict gene expression [1]. Its first task is a three-class classification problem with low, medium, and high expression labels. The trained representation is then used for a regression task that predicts a numerical expression value.

The complete method in the paper starts by pretraining a standard BERT model from scratch. The reported architecture has 12 hidden layers, 12 attention heads, and about 110 million parameters. The paper also compares CodonBERT with simpler baseline models, including Random Forest and LSTM. We did not repeat these baseline models because our main goal was to study and reproduce the CodonBERT workflow.

Training a model of this size from scratch would require long and stable GPU access. The best free resource available to us was a Tesla T4 GPU in Google Colab, and long

sessions were difficult to keep connected. For this reason, we used the public `lhal1ee/CodonBERT` checkpoint from Hugging Face [4]. This checkpoint contains weights from Sanofi's CodonBERT model, which was pretrained on a large collection of mRNA sequences [5]. We then reproduced the downstream classification and regression idea of the target paper.

2 Reviewed Paper and Project Scope

The target paper uses the structure of sentiment analysis for gene expression prediction. In ordinary sentiment analysis, samples may be separated into negative, neutral, and positive classes. In the paper, genes are instead separated into low, medium, and high expression classes. BERT learns a sequence representation from this first task. After that, the model is changed to a regression model to estimate a continuous expression score.

Our work follows this two-stage idea for one of the organisms used in the paper, *S. cerevisiae*. We first fine-tuned CodonBERT as a classifier. We then removed the classification head, added a regression head, froze the first six transformer layers, and fine-tuned the remaining part of the model.

This project should not be described as an exact reproduction of the original model. The target paper trained its own CodonBERT, while our notebook starts from a different pretrained checkpoint. Therefore, our results show a reproduction of the main downstream approach under limited compute, not a direct rerun of every experiment in the paper.

3 Data Preparation

We used two data sources for baker's yeast. The coding sequences came from the *Saccharomyces cerevisiae* S288C reference, assembly R64-1-1, from Ensembl Genomes [3]. The raw gene expression counts came from Expression Atlas experiment E-MTAB-8626 [2].

The raw expression table contained 7,127 rows and 27 sample columns. We converted the sample columns to numerical values and used the median count across samples as `expression_raw` for each gene. The CDS file contained 6,600 sequences. We kept sequences that started with

ATG, ended with a valid stop codon, and had a length divisible by three. This left 6,590 sequences. After joining the sequence and expression tables by `gene_id`, the final dataset contained 6,030 genes.

For classification, we transformed the raw expression value using

$$x_{\log} = \log(1 + x_{\text{raw}}). \quad (1)$$

We calculated the 33rd and 67th percentiles of this value. Genes below the first limit received label 0, genes between the two limits received label 1, and genes above the second limit received label 2. The final class counts were 1,987, 2,049, and 1,994, so the classes were close to balanced.

Each RNA sequence was divided into groups of three nucleotides. Spaces were added between codons so that the tokenizer could read every codon as one token. Tokenization first produced sequences with a maximum length of 1,064. Before training, the sequences were limited to 1,024 tokens because this is the maximum input length supported by the checkpoint used in our notebook.

4 Model and Training

4.1 Classification Stage

We loaded the pretrained CodonBERT backbone and added dropout with a rate of 0.1 and a linear classification layer on the [CLS] representation. Cross-entropy loss was used for the three classes. The dataset was divided into 5,125 training genes and 905 validation genes with a stratified split.

The classifier was trained for three epochs with a learning rate of 2×10^{-5} and weight decay of 0.01. The batch size was 2, and eight gradient accumulation steps were used to reduce GPU memory use. Mixed-precision training was enabled. The model was evaluated and saved every 200 steps, and the checkpoint with the best macro F1-score was loaded at the end.

4.2 Regression Stage

For regression, the 6,030 genes were split into 4,220 training, 905 validation, and 905 test samples. The target was transformed with `log1p` and scaled to the interval $[-1, 1]$. The classifier’s CodonBERT backbone was reused, the classification head was replaced with a linear regression head, and the first six transformer layers were frozen. This left 43.96 million trainable parameters out of 86.49 million total parameters.

The regression model was trained for five epochs using mean squared error loss. The learning rate, batch size, gradient accumulation, weight decay, and mixed-precision settings were the same as in the classification stage. The best checkpoint was selected using validation Spearman correlation.

Table 1: Classifier results on the validation set.

Metric	Value
Accuracy	0.6652
Macro precision	0.6699
Macro recall	0.6657
Macro F1-score	0.6673
Weighted F1-score	0.6665
AUROC (one-vs-rest)	0.8311

Table 2: Per-class validation results.

Class	Precision	Recall	F1	Support
0 (low)	0.6950	0.6577	0.6759	298
1 (mid)	0.5723	0.6169	0.5938	308
2 (high)	0.7423	0.7224	0.7322	299

5 Results

5.1 Classification Results

Table 1 presents the final validation metrics. The model reached an accuracy of 0.665 and a macro F1-score of 0.667. Class 2 had the highest F1-score, while class 1 was the most difficult class. This is also visible in the confusion matrix, where more medium-expression genes were confused with the low and high classes.

The reviewed paper reports an AUROC of 0.81, accuracy of 0.62, precision of 0.62, and recall of 0.62 for its CodonBERT classifier. Our values are close and are slightly higher for the reported measures. However, the comparison is only approximate because the pretrained model and the exact evaluation procedure are different. We therefore cannot say that our model is strictly better than the paper’s model.

5.2 Regression Results

On the regression test split, the model reached an MSE of 0.0284 and an MAE of 0.1142 in the scaled target space. Spearman correlation was 0.8632. The same Spearman value was obtained after converting the predictions back to raw expression values because this conversion keeps the ordering of the samples.

The paper reports a Spearman value of about 0.63 for this organism. Our value is higher, but it is not a direct reproduction result. A possible reason is the stronger pretrained checkpoint. More importantly, the two training stages used separate random splits, which means that some genes in the regression test set may already have been seen during classifier training. This overlap can increase the regression score.

Table 3: Regression results on the test split.

Metric	Value
Scaled MSE	0.0284
Scaled MAE	0.1142
Spearman correlation	0.8632

6 Discussion and Limitations

The results show that codon-level representations can be useful for gene expression prediction. The classifier performs much better than random prediction, and the regression model gives a strong ranking of genes by expression level. The middle class is harder to predict than the low and high classes because its samples are close to both percentile limits.

There are several limits to this project. First, we used pretrained weights from a different CodonBERT project instead of pretraining the target paper’s model from scratch. Second, only the yeast dataset was tested, while the paper studied six organisms. Third, we did not repeat the Random Forest, LSTM, and other baseline experiments. Fourth, the classification stage has a validation set but no separate test set. Fifth, label limits and regression scaling values were calculated before the data splits. Finally, the independent classification and regression splits allow overlap between classifier training samples and the later regression test set. A stricter reproduction should create one fixed train, validation, and test split before any label calculation or model training and keep the test genes unseen in both stages.

These limits do not remove the educational value of the project, but they affect how the results should be described. The project demonstrates an adapted and compute-limited reproduction of the paper’s method. It does not prove a better result than the original study.

7 Conclusion

In this project, we reviewed a CodonBERT paper and reproduced its main two-stage idea for yeast gene expression. We prepared 6,030 matched coding sequences and expression values, trained a three-class expression classifier, and then fine-tuned the backbone for regression. The final classifier reached a macro F1-score of 0.667, and the regression model reached a Spearman correlation of 0.863.

The experiment shows that a pretrained codon language model can learn useful information for both expression classes and expression ranking. At the same time, the differences in pretraining and data splitting mean that the reported values should be treated as results from our adapted implementation. Future work should use one fixed split from the beginning, remove all test-set overlap, repeat the baseline models, and then test the same procedure on the other organisms used in the paper.

References

- [1] Ashley Nicole Babjac, Zhixiu Lu, and Scott J. Emrich. CodonBERT: Using BERT for sentiment analysis to better predict genes with low expression. In *Proceedings of the 14th ACM International Conference on Bioinformatics, Computational Biology and Health Informatics*. Association for Computing Machinery, 2023.
- [2] EMBL-EBI. Expression atlas experiment E-MTAB-8626. Expression Atlas. Accessed 2026-09-02.
- [3] Ensembl Genomes. *Saccharomyces cerevisiae* S288C coding sequences, assembly R64-1-1, release 56. Ensembl Fungi. Accessed 2026-09-02.
- [4] lhallee. CodonBERT model weights. Hugging Face. Accessed 2026-09-02.
- [5] Sizhen Li, Saeed Moayedpour, Ruijiang Li, et al. CodonBERT: Large language models for mRNA design and optimization. *bioRxiv*, 2023.